Artificial Intelligence

(part 4c)

Strategies for

State Space Search

(Informed..Heuristic search)



9	1						4	5
		6	5		1	3		
5								9
		8	9		3	2		
	7						5	
		4	8		7	1		
8								6
		5	3		6	9		
6	4						2	1

Search Strategies (The Order..)

- Uninformed Search
  - breadth-first
  - depth-first
  - iterative deepening
  - uniform-cost search
  - depth-limited search
  - bi-directional search
  - constraint satisfaction

- Informed Search
  - best-first search
  - search with heuristics
  - memory-bounded search
  - iterative improvement search

HEURISTIC SEARCH

•(rules of thumb): Weak search method because it is based on experience or intuition.

•Have long been a core concern in AI research

•Used to prune spaces of possible solution

• When to employ Heuristic?

1. A problem may not have an exact solution.
   - e.g. medical diagnosis: doctors use heuristic
2. A problem may have an exact solution, but the computational cost of finding it may be prohibitive.
   - e.g in chess (exhaustive or brute-force search)

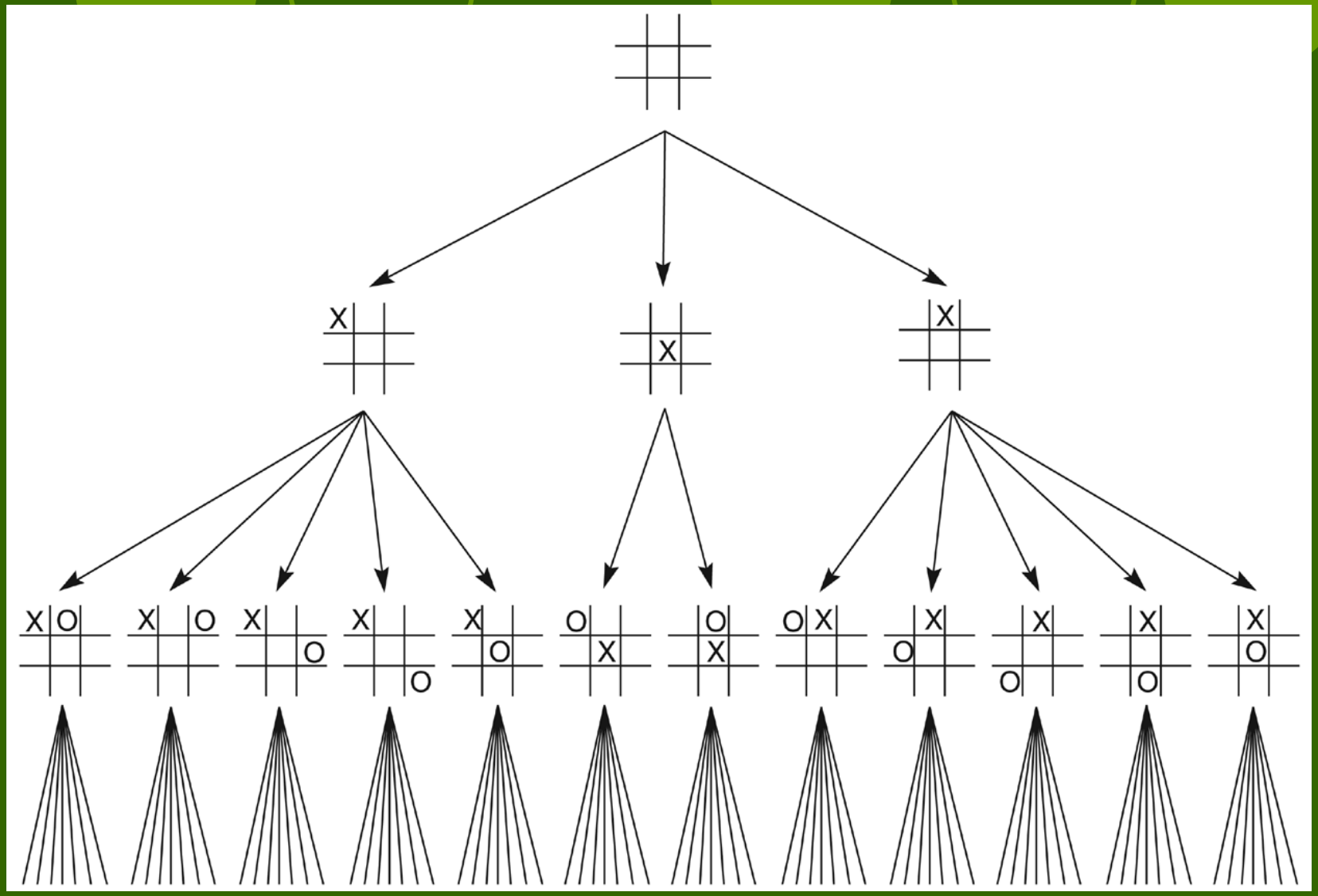
brute-force search

- In computer science, a brute-force search consists of systematically enumerating every possible solution of a problem until a solution is found, or all possible solutions have been exhausted.
- For example, an anagram problem can be solved by enumerating all possible combinations of words with the same number of letters as the desired phrase, and checking one by one whether the words make a valid anagram.

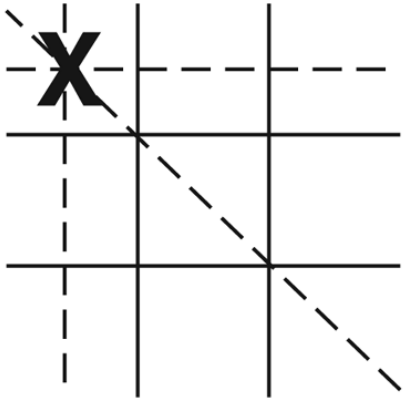
anagram

- A word that is spelled with the exact same letters as another word. Example: RIDES is an anagram of SIRED and vice versa

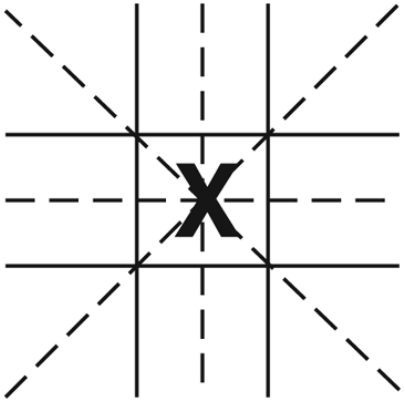
Eg. To Reduce search=> First three levels of the tic-tac-toe state space reduced by symmetry (simple heuristic-most winning opportunities)



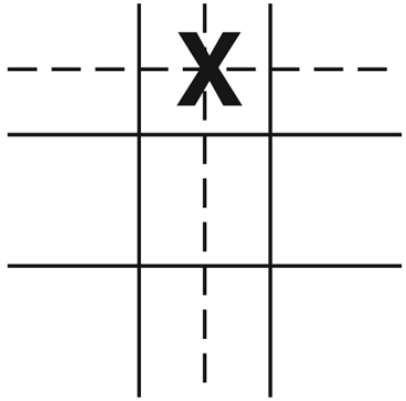
The “most wins” heuristic applied to the first children in tic-tac-toe.



Three wins through a corner square

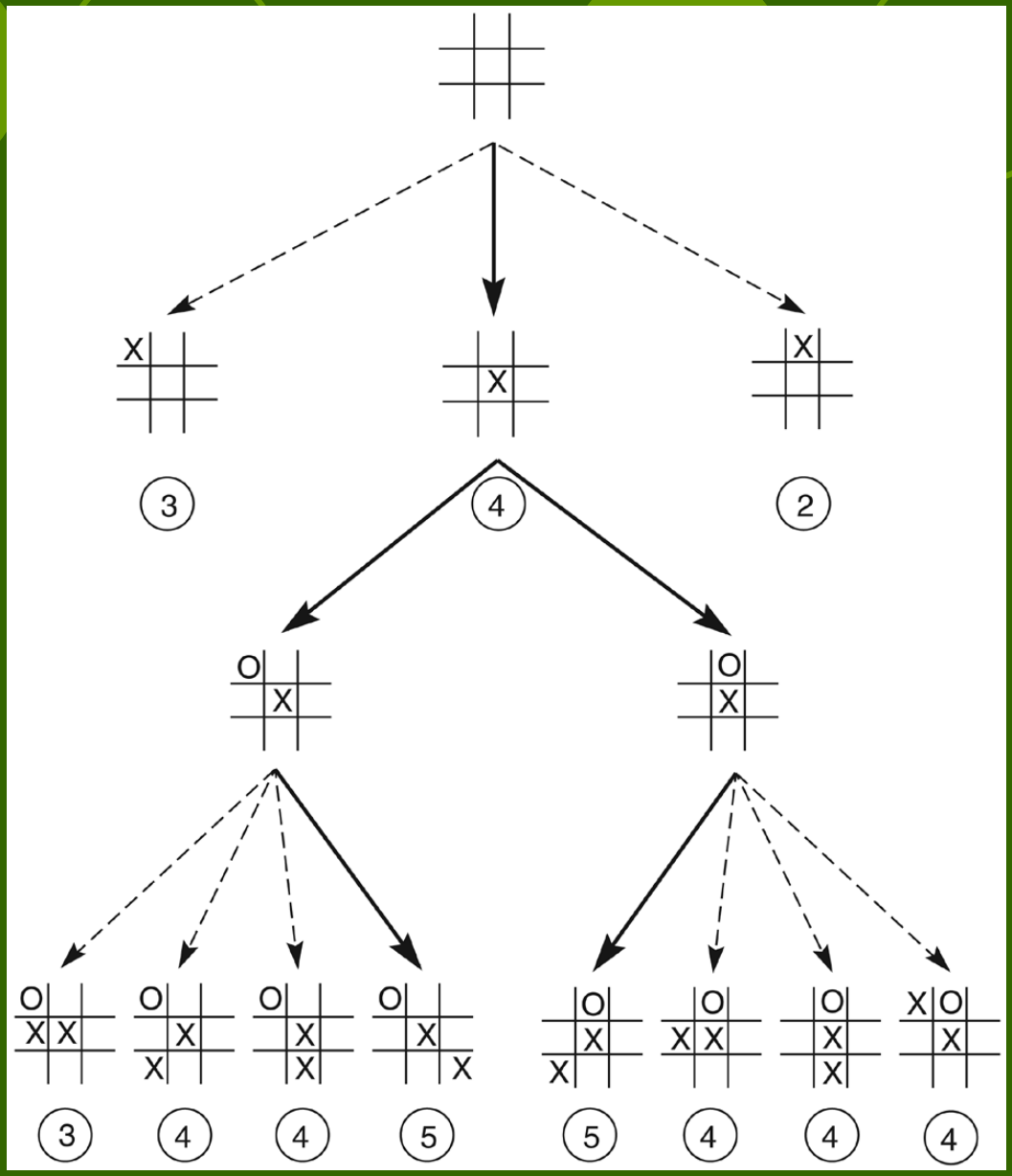


Four wins through the center square



Two wins through a side square

Heuristically reduced state space for tic-tac-toe.



HEURISTIC SEARCH

- HEURISTIC SEARCH (rules of thumb)
- Can be viewed as two parts:

  -the heuristic measure

  - an algorithm that uses it
- An Algorithm for heuristic search: HILL CLIMBING

HEURISTIC SEARCH: HILL CLIMBING

- simplest, the best child is selected for further expansion
- limited memory, no backtracking and recovery

- Problem with hill climbing:
  - An erroneous heuristic can lead along an infinite paths that fail.
  - Can stuck at local maxima – reach a state that is better evaluation than its children, the algorithm halts.
  - There is no guarantee optimal performance
- Advantage:-
  - Can be used effectively if the heuristic is sufficient

HEURISTIC SEARCH: BEST-FIRST SEARCH

- It is a general algorithm for heuristically searching any state space graph
- Supports a variety of heuristic evaluation functions

HEURISTIC SEARCH: BEST-FIRST SEARCH

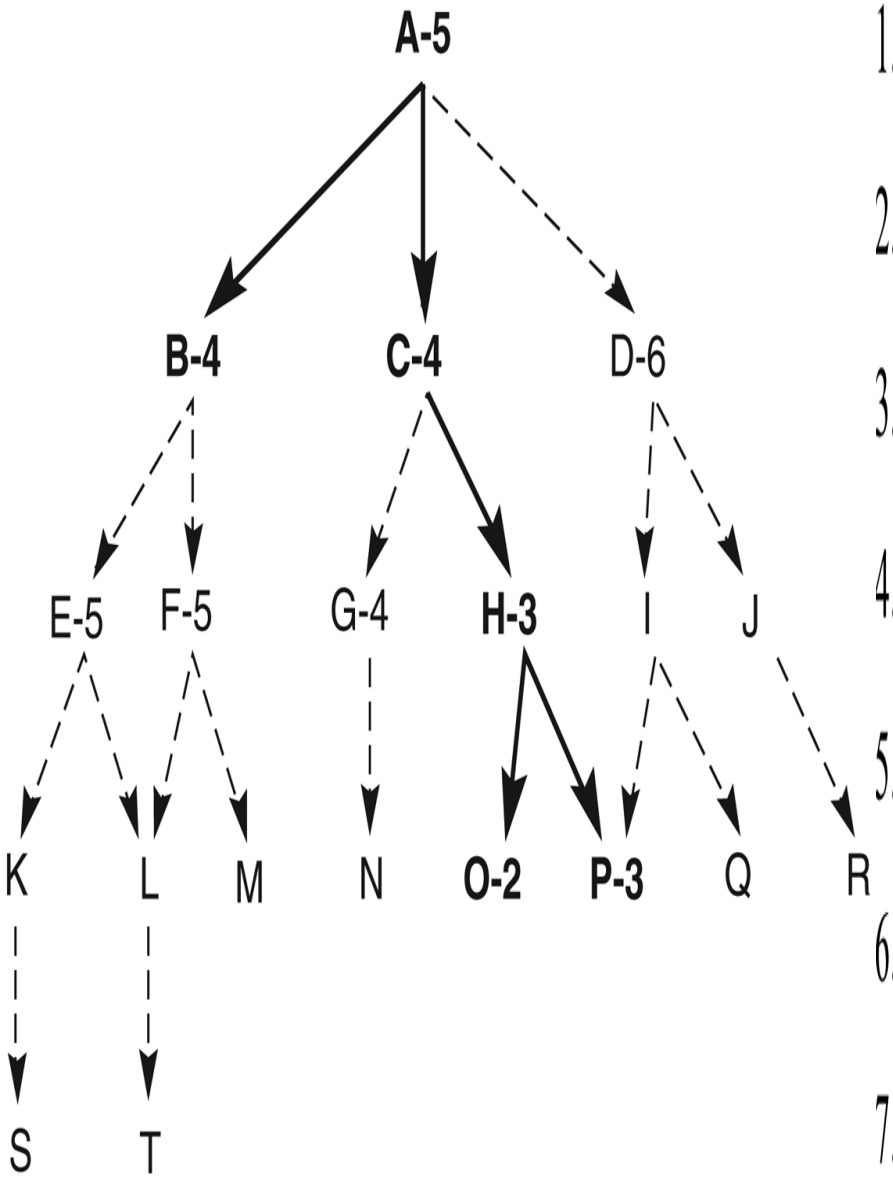
- Better and flexible Algorithm for heuristic search
- BEST-FIRST SEARCH:
  - Avoid local maxima, dead ends; has open and close lists
  - selects the most promising state
  - apply heuristic and sort the ‘best’ next state in front of the list (*priority queue*) – can jump to any level of the state space
  - If lead to incorrect path, it may retrieve the next best state

function best_first_search algorithm

```
function best_first_search;

begin
   open := [Start];                                                    % initialize
   closed := [ ];
   while open ≠ [ ] do                                                 % states remain
      begin
         remove the leftmost state from open, call it X;
         if X = goal then return the path from Start to X
         else begin
                  generate children of X;
                  for each child of X do
                  case
                        the child is not on open or closed:
                           begin
                                 assign the child a heuristic value;
                                 add the child to open
                           end;
                        the child is already on open:
                           if the child was reached by a shorter path
                           then give the state on open the shorter path
                        the child is already on closed:
                           if the child was reached by a shorter path then
                              begin
                                 remove the state from closed;
                                 add the child to open
                              end;
                  end;                                                 % case
                  put X on closed;
                  re-order states on open by heuristic merit (best leftmost)
               end;
return FAIL                                                            % open is empty
end.
```

Heuristic search of a hypothetical state space.



1. open = [A5]; closed = [ ]
2. evaluate A5; open = [B4,C4,D6]; closed = [A5]
3. evaluate B4; open = [C4,E5,F5,D6]; closed = [B4,A5]
4. evaluate C4; open = [H3,G4,E5,F5,D6]; closed = [C4,B4,A5]
5. evaluate H3; open = [O2,P3,G4,E5,F5,D6]; closed = [H3,C4,B4,A5]
6. evaluate O2; open = [P3,G4,E5,F5,D6]; closed = [O2,H3,C4,B4,A5]
7. evaluate P3; the solution is found!

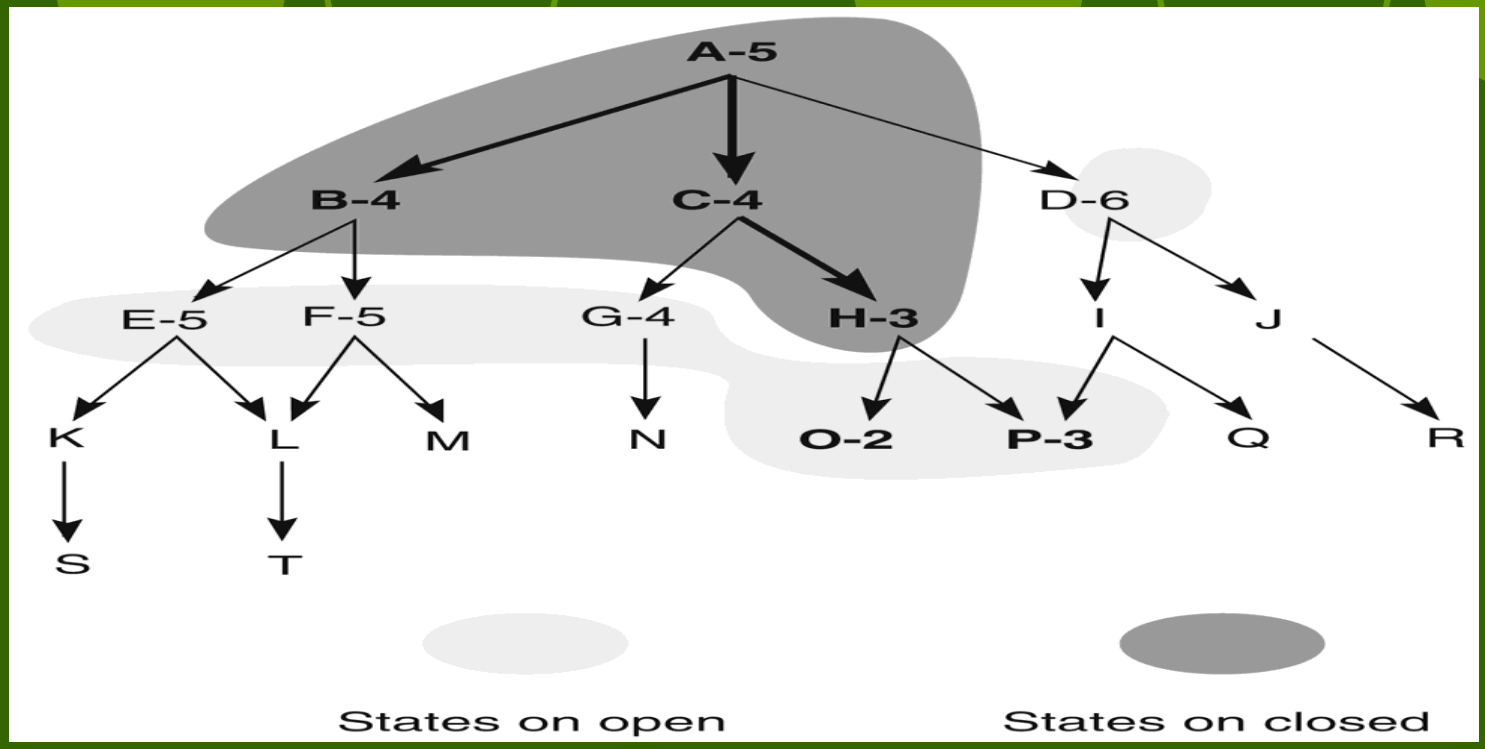
A trace of the execution of best_first_search for Figure 4.4

1. **open = [A5]; closed = [ ]**
2. **evaluate A5; open = [B4,C4,D6]; closed = [A5]**
3. **evaluate B4; open = [C4,E5,F5,D6]; closed = [B4,A5]**
4. **evaluate C4; open = [H3,G4,E5,F5,D6]; closed = [C4,B4,A5]**
5. **evaluate H3; open = [O2,P3,G4,E5,F5,D6]; closed = [H3,C4,B4,A5]**
6. **evaluate O2; open = [P3,G4,E5,F5,D6]; closed = [O2,H3,C4,B4,A5]**
7. **evaluate P3; the solution is found!**

Q1: open nodes to visit are sorted in what order?

Q2: closed nodes?

Figure 4.5: Heuristic search of a hypothetical state space with open and closed states highlighted.



1. **open = [A5]; closed = [ ]**
2. **evaluate A5; open = [B4,C4,D6]; closed = [A5]**
3. **evaluate B4; open = [C4,E5,F5,D6]; closed = [B4,A5]**
4. **evaluate C4; open = [H3,G4,E5,F5,D6]; closed = [C4,B4,A5]**
5. **evaluate H3; open = [O2,P3,G4,E5,F5,D6]; closed = [H3,C4,B4,A5]**
6. **evaluate O2; open = [P3,G4,E5,F5,D6]; closed = [O2,H3,C4,B4,A5]**
7. **evaluate P3; the solution is found!**

HEURISTIC EVALUATION FUNCTION f(n)

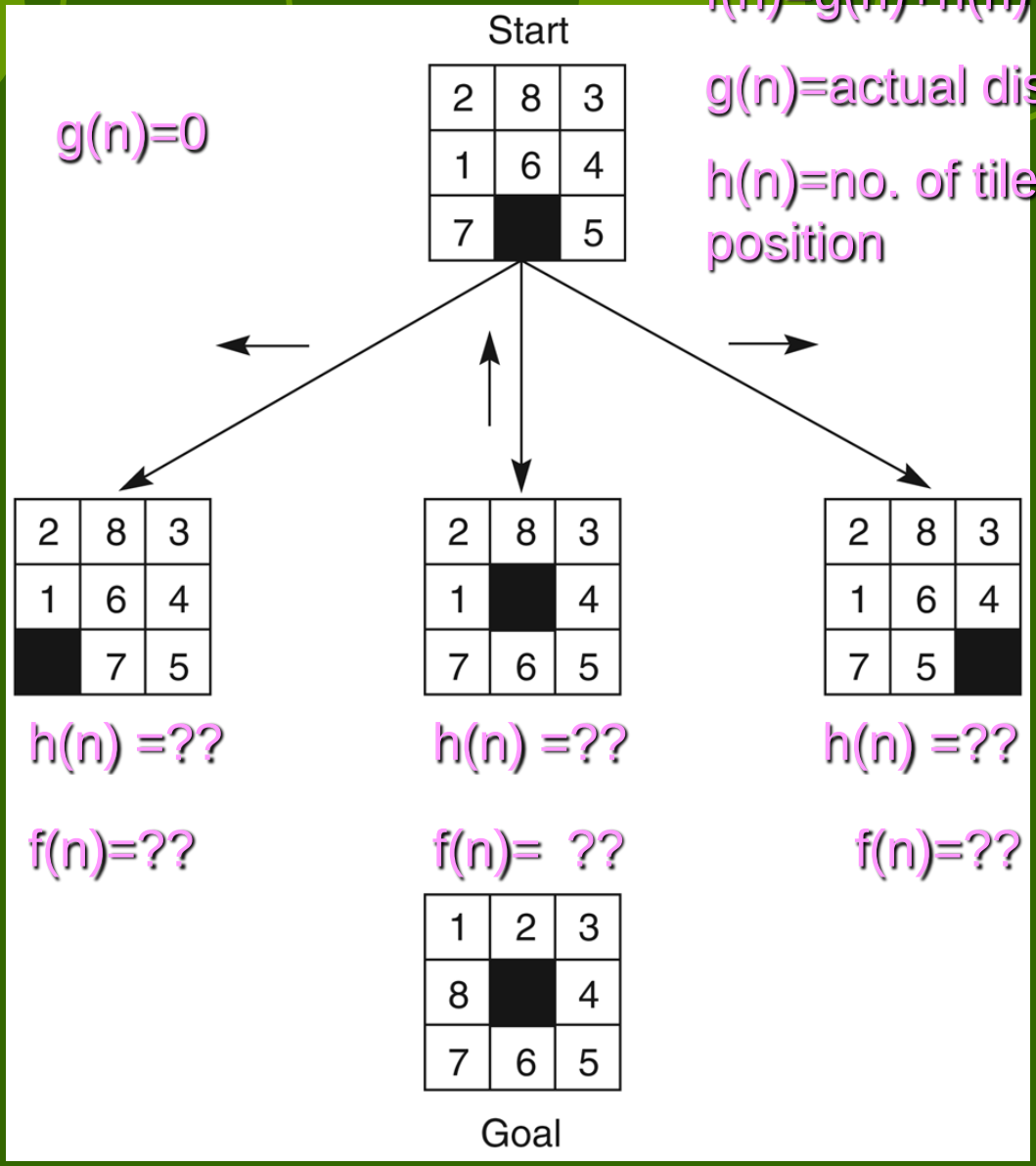
- To evaluate performances of heuristics for solving a problem.
- Devise good heuristic using limited information to make intelligent choices.
- To better heuristic, f(n)=g(n)+h(n), where h(n) distance from start to n, g(n) is distance from n to goal
- Eg. 8-puzzle, heuristics h(n) could be:
  - No. of tiles in wrong position
  - No. of tiles in correct position
  - Number of direct reversal (2X)
  - Sum of distances out of place
- And g(n) is the depth measure

The start state, first set of moves, and goal state for an 8-puzzle instance.

f(n)=g(n)+h(n)

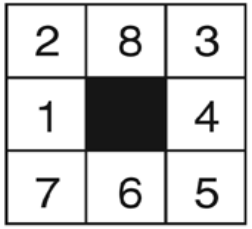
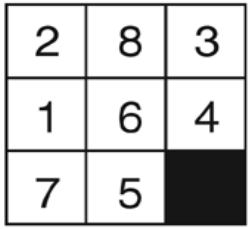
g(n)=actual dist. From n to start

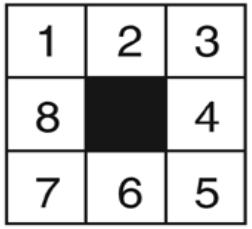
h(n)=no. of tiles in wrong position



Three heuristics applied to states in the 8-puzzle.

-Devising good heuristics is sometimes difficult; OUR GOAL is to use the limited information available to make INTELLIGENT CHOICE

	**5**	**6**
	**3**	**4**
	**5**	**6**
	Tiles out of place	Sum of distances out of place



Goal

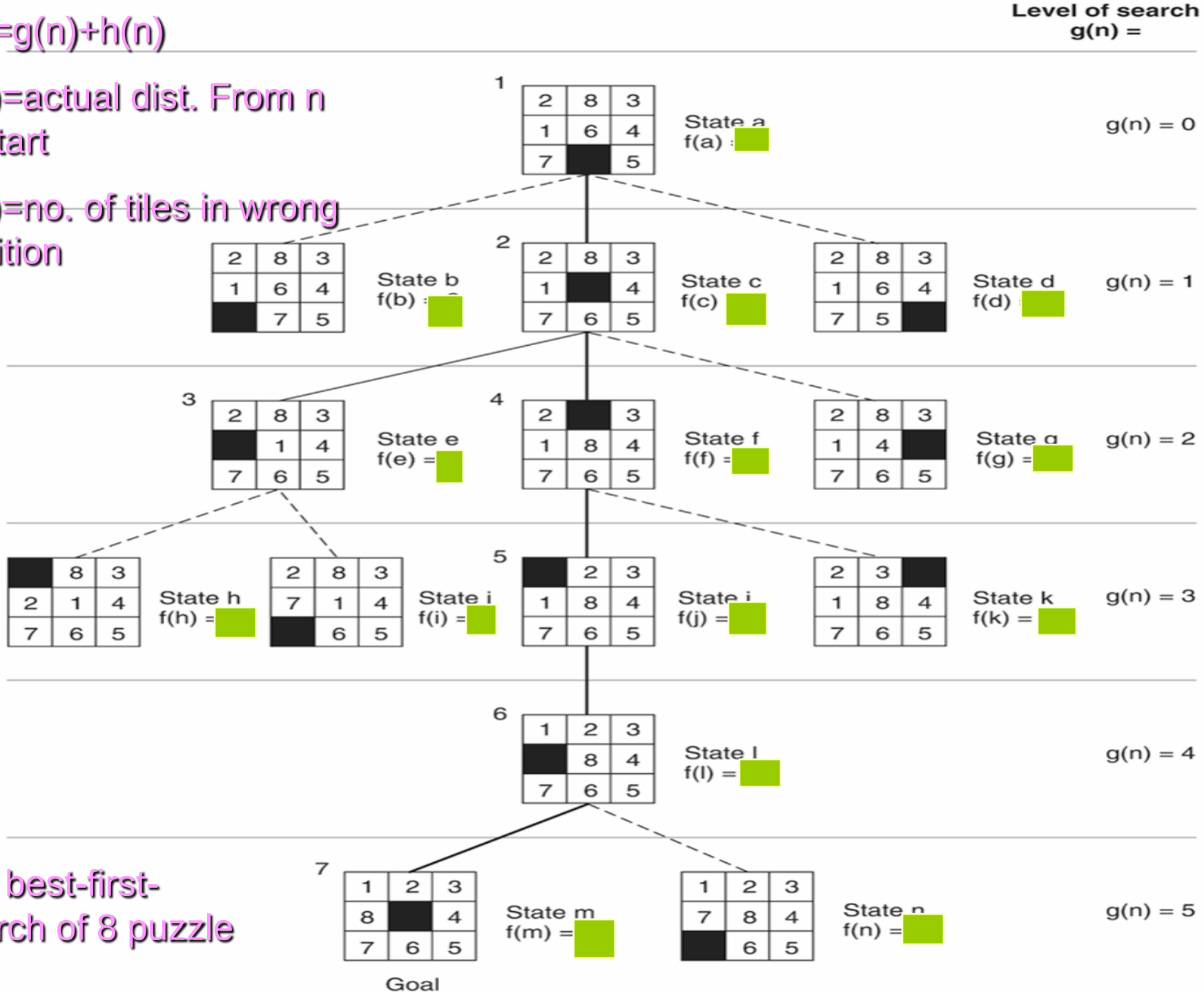
POP –QUIZ (in pairs)

- In the tree of 8-puzzle given in the next slide, Give the value of f(n) for each state, based on g(n) and h(n)
- Trace using best-first-search, what will be the lists of open and closed states?

f(n)=g(n)+h(n)

g(n)=actual dist. From n to start

h(n)=no. of tiles in wrong position

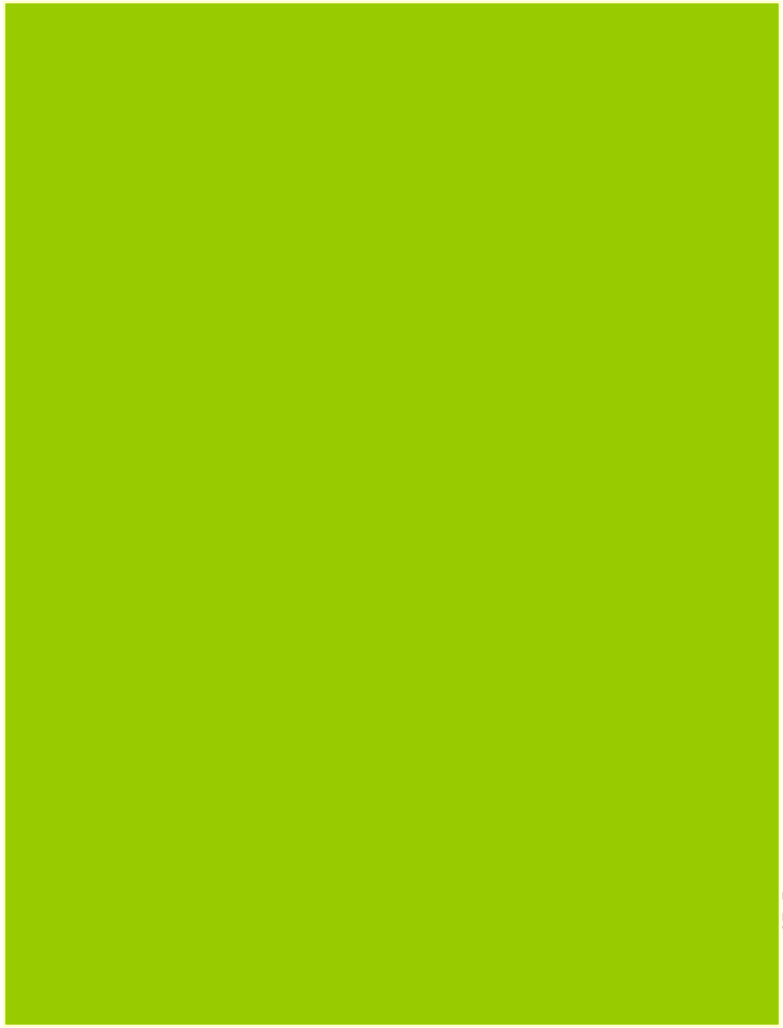


Full best-first-search of 8 puzzle

State space generated in heuristic search of the 8-puzzle graph.

The successive stages of **open** and **closed** that generate previous graph are:

1. **open = [a4];**
   closed = [ ]
2. **open =**
   closed
3. **open =**
   closed
4. **open =**
   closed
5. **open =**
   closed
6. **open =**
   closed
7. **open =**
   closed
8. **success, m = goal!**





open and **closed** as they appear after the third iteration of heuristic search.